Accelerating Static Timing Analysis from One Hour to One Minute

A Tunable Algorithm for Large-Scale Hardware Designs

Anonymous Author(s)*

Abstract

Verifying timing requirements is fundamental to ensuring correct functionality and performance of modern chip designs. *Static timing analysis* (STA) is the industry-standard technique for this task, enabling early detection of circuit paths that violate timing requirements. To achieve *high-precision* in STA reports, modern STA tools widely adopt *common path pessimism removal* (CPPR) techniques. However, existing algorithms for high-precision STA with CPPR incur substantial computational overhead: they are typically 10×–100× slower than STA without CPPR and scale poorly with circuit size.

This paper presents FlashTimer, a novel STA algorithm that can achieve efficiency comparable to STA without CPPR while retaining the full precision of STA with CPPR. FlashTimer accelerates STA by drastically reducing unnecessary path exploration, improving the worst-case complexity of existing approaches from exponential or quadratic to nearly linear. Additionally, FlashTimer allows for tunable precision–performance trade-off, allowing users to sacrifice a small amount of precision for further substantial speedups.

Our evaluation demonstrates that FlashTimer significantly outperforms state-of-the-art open-source STA tools with CPPR, achieving up to 837× speedup (59× on average)—reducing hour-long analyses to minutes—while preserving full precision. With only a deliberate 1.4% precision scrifice, FlashTimer can attain speedups of up to 1773× (113× on average). Our further scalability experiments show that FlashTimer's runtime grows approximately linearly with circuit size. These results establish FlashTimer as a highly efficient and scalable solution for high-precision STA with CPPR on large-scale chip designs.

CCS Concepts: • **Hardware → Static timing analysis**.

Keywords: Common Path Pessimism Removal

1 Introduction

Modern high-performance hardware designs operate at clock frequencies exceeding the gigahertz scale, where even timing violations at nanosecond scale can compromise correct functionality and performance [3, 5]. If such violations escape detection before tape-out, they may necessitate costly silicon respins, leading to millions of dollars in additional manufacturing costs and months of project delay [1, 21].

Static timing analysis (STA) serves as the industry-standard safeguard against timing violations during hardware design [8]. Given a circuit, STA aims to identify and report circuit paths that violate timing constraints. In practice, developers typically query the top-k most critical timing-violating paths, fix them, and then rerun STA to obtain the next set of critical violations [12, 13]. This iterative workflow, commonly referred to as *timing closure* [7], arises because timing fixes—such as buffer insertion, register retiming, or combinational logic optimization—modify the circuit structure and consequently change the timing analysis results.

To achieve *high precision* in STA reports, modern STA tools widely adopt *common path pessimism removal* (CPPR) techniques [10]. The precision improvement arises from *removing* the over-conservatism introduced when the maximum delay and minimum delay of a *common path* segment are *simultaneously* assumed during timing analysis. The difference between the maximum and minimum delay is referred to as *pessimism*. Such simultaneous assumptions are physically impossible, as a circuit path cannot exhibit both its maximum and minimum delay at the same time. By removing this pessimism, CPPR produces more accurate timing estimates and reduces false timing violation reports.

While CPPR substantially improves STA precision, it also significantly increases analysis runtime. In practice, STA with CPPR is typically 10×–100× slower than STA without CPPR [11**?**]. To better understand the source of this overhead, we categorize existing algorithms [6, 11, 14–20, 23] based on how CPPR interacts with critical path search:

- *CPPR Before Search* [11, 14–16]: These approaches first perform global pessimism removal prior to critical path search. This global CPPR phase requires quadratic time in the number of registers [11]. Afterward, the top-k critical paths can be efficiently computed in linearithmic time [9]. Consequently, the overall runtime is dominated by the quadratic global CPPR phase, limiting scalability for large-scale designs with millions of registers.
- *CPPR During Search* [6, 17–20, 23]: These approaches integrate pessimism removal into the critical path search procedure. In the worst case, such on-the-fly CPPR requires enumerating a combinatorial number of paths, leading to exponential time complexity. Although heuristic pruning in prior work can reduce practical runtime, the underlying worst-case complexity remains exponential, resulting in inherent scalability limitations for large designs.

In this paper, we present FlashTimer, a novel STA algorithm that can achieve efficiency comparable to STA without CPPR while retaining the full precision of STA with CPPR. The key insight of FlashTimer is a new paradigm that we term *CPPR After Search*:

1. *Search Phase*: FlashTimer performs a top-$(k + \alpha)$ critical path search in *linearithmic time* [9], where k is the number of queried critical paths and α is a small redundancy factor that safely over-approximates the post-CPPR results.
2. *CPPR Phase*: FlashTimer then applies CPPR only to these $k + \alpha$ candidate paths and then select the top-k critical ones. For each path, its CPPR takes *constant time* [13].

Because k is a user-specified constant that is typically much smaller than the circuit size [12, 13], and we demonstrate both theoretically (Section 5) and empirically (Section 6) that α can be treated as a small constant, the overall complexity of FlashTimer is *linearithmic* in the circuit size. This represents a substantial improvement over prior approaches, which incur either *quadratic* (CPPR Before Search) or *exponential* (CPPR During Search) worst-case complexity. As a result, FlashTimer achieves significantly better efficiency and scalability, particularly for large-scale hardware designs.

In addition, FlashTimer is tunable, enabling an explicit trade-off between precision and performance. Users may sacrifice a small amount of precision to obtain further speedups. This tunability is controlled by the redundancy factor α. By default, FlashTimer automatically estimates a safe value of α on the fly to preserve full precision. This estimated value is typically conservative—i.e., slightly larger than the minimal α for optimal efficiency—while still delivering substantial performance gains over existing approaches. For users who prioritize performance over strict precision, α can be manually fixed to a smaller value, enabling additional speedups at the cost of minor precision loss.

We evaluate FlashTimer on two benchmark suites from two renowned ACM-sponsored Tau Contests [12, 13] focused on STA with CPPR. These large-scale benchmarks include circuits with up to 3.84 million nets. We compare against two state-of-the-art open-source STA tools with CPPR: the CPPR-Before-Search approach *OpenTimer* [14, 15] and the CPPR-During-Search approach *TKTimer* [18].

The results demonstrate substantial performance improvements. Compared to the baselines, FlashTimer achieves up to 837× speedup (59× on average), reducing hour-long analyses to minutes while preserving full precision. With a deliberate 1.4% precision sacrifice, FlashTimer attains speedups of up to 1773× (113× on average). Additional scalability experiments show that FlashTimer's runtime grows nearly linearly with circuit size. These results establish FlashTimer as a highly efficient and scalable solution for STA with CPPR on large-scale designs.

In summary, this paper makes the following contributions:

- We present FlashTimer, a novel STA algorithm that can achieve efficiency comparable to STA without CPPR while retaining the full precision of STA with CPPR.
- We formally prove the correctness and time complexity of FlashTimer, establishing a theoretical foundations of the *CPPR-After-Search* paradigm for high-precision STA.
- We conduct an extensive experimental evaluation demonstrating that, compared to state-of-the-art approaches, FlashTimer achieves an average 59× speedup in full-precision mode and up to 113× speedup with only a deliberate 1.4% precision sacrifice.
- We will open-source FlashTimer and its experimental artifacts to ensure reproducibility and support future research.

We believe the CPPR-After-Search paradigm opens new directions for scalable high-precision STA beyond FlashTimer itself and hope our work can inspire future STA innovations.

2 Background

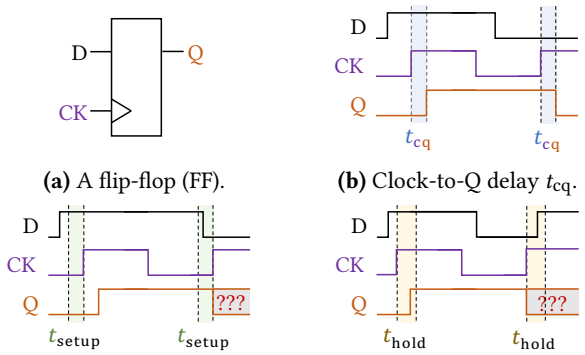
This section provides the necessary background for our discussion about FlashTimer, including preliminaries on *static timing analysis* (STA) (Section 2.1) and *common path pessimism removal* (CPPR) (Section 2.2).

2.1 Static Timing Analysis

STA aims to assist hardware designers in identifying and eliminating timing violations in synchronous circuits. To formally describe the STA problem, we first introduce the timing model of hardware registers, including the *timing parameters* that characterize their timing behavior (Section 2.1.1) and the *timing constraints* they impose on circuit design (Section 2.1.2). With these preliminaries, we then formulate STA as a graph-theoretic problem (Section 2.1.3). For clarity of exposition, we assume synchronous circuits in which all registers are *positive-edge-triggered* flip-flops (FFs).

2.1.1 Timing Parameters. For an FF shown in Figure 1a, there are three fundamental timing parameters (whose concrete values are typically specified in the standard cell library provided to STA tools by the technology vendor):

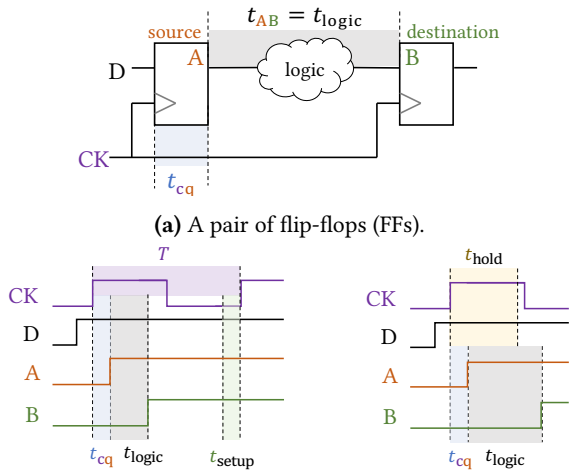
- t_{cq}—*clock-to-Q delay*—denotes the time interval from the rising edge of the clock signal *CK* to the moment when the data appears at the output Q, shown in blue in Figure 1b.
- t_{setup}—*setup time*—is the minimum time interval for which the input data D must arrive *before* the rising clock edge to ensure correct sampling at Q, illustrated in green in Figure 1c. If D arrives within t_{setup} before the clock edge (i.e., too late), correct sampling is no longer guaranteed, as indicated by the question mark in Figure 1c.
- t_{hold}—*hold time*—is the minimum time interval for which the input data D must remain stable *after* the rising clock edge to ensure correct sampling at Q, illustrated in yellow in Figure 1d. If D changes within t_{hold} after the clock edge (i.e., too early), correct sampling is not guaranteed, as indicated by the question mark in Figure 1d.



(a) A flip-flop (FF). **(b)** Clock-to-Q delay t_{cq}.

(c) Setup time t_{setup}. **(d)** Hold time t_{hold}.

Figure 1. A positive-edge-triggered FF and waveforms illustrating the semantic of its fundamental timing parameters.



(a) A pair of flip-flops (FFs).

(b) Setup constraint. **(c)** Hold constraint.

Figure 2. A register-to-register path and waveforms illustrating the timing constraints it imposes on the circuit.

2.1.2 Timing Constraints.

Consider an arbitrary pair of flip-flops (FFs), as shown in Figure 2a, where data propagates from a source FF to a destination FF through combinational logic. STA aims to enforce two fundamental timing constraints on such register-to-register paths:

- *Setup constraint* requires that the data D, launched by the current clock edge, arrive at B sufficiently early—i.e., no later than t_{setup} before the *next* clock edge—so that it can be correctly sampled by that clock edge. As illustrated in Figure 2b, this can be formalized as:

$$t_{cq} + t_{logic} + t_{setup} \leq T, \tag{1}$$

  where T is the clock period and t_{logic} is the propagation delay of the combinational logic between A and B.
- *Hold constraint* requires that the newly launched data must not arrive at B too early, ensuring that the current value at B remains stable for at least t_{hold} after the *current* clock edge, so that it can be correctly sampled by that clock edge. As illustrated in Figure 2c, this can be formalized as:

$$t_{cq} + t_{logic} \geq t_{hold}. \tag{2}$$

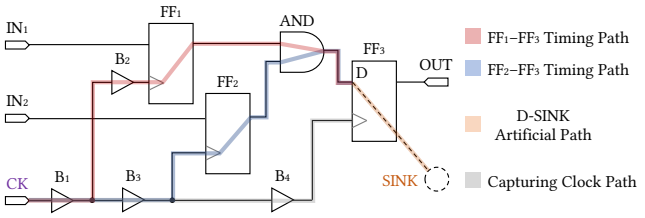


Figure 3. An example circuit graph for static timing analysis.

In real-world STA practice, timing scenarios are more complex for two primary reasons: (1) the clock signal may not arrive at all FFs simultaneously due to clock latency; (2) the delay of each digital component is affected by process, voltage, temperature (PVT), and other environmental variations. Consequently, practical timing constraints are more sophisticated than the idealized forms in Formula 1 and 2. For clarity of exposition, we focus our later discussion on the hold constraint. Detecting setup violations and hold violations are dual problems and can be addressed in a symmetric manner.

To account for clock latency (as illustrated in Figure 3), the hold constraint Formula 2 must be refined as:

$$t_{data} \geq t_{hold} + t_{clock}. \tag{3}$$

Here, t_{data} denotes the data arrival time, which equals $t_{cq} + t_{logic}$ in the ideal case shown in Figure 2a. In contrast, for the more complex FF_1–FF_3 timing path from CK to D in Figure 3, the data arrival time t_{data} is $t_{B_1} + t_{B_2} + t_{cq} + t_{AND}$. Moreover, the clock edge from CK does not instantaneously arrive at FF_3 as the ideal case shown in Figure 2c; it experiences a propagation delay, denoted by t_{clock}, which is why this term appears on the right-hand side of Formula 3. For both the FF_1–FF_3 and FF_2–FF_3 timing paths in Figure 3, we have $t_{clock} = t_{B_1} + t_{B_3} + t_{B_4}$, since both paths share the same capturing clock path from CK to the destination FF_3.

To model environmental variations, the delay of each digital component is represented as an interval bounded by a minimum and a maximum delay in the standard cell library, rather than as a single deterministic value. To guarantee that Formula 3 holds under all circumstances, traditional STA conservatively verifies the following condition:

$$\min(t_{data}) \geq \max(t_{hold}) + \max(t_{clock}). \tag{4}$$

Here, $\max(t_{hold})$ is a constant independent of the circuit structure, whereas $\min(t_{data})$ and $\max(t_{clock})$ depend on the specific timing path and circuit topology.

2.1.3 Problem Formulation.

STA aims to report the top-k most critical timing-violating paths. To quantify how critically a path violates the hold constraint, the STA community

introduces the notion of *slack*, defined as

$$slack = \min(t_{\text{data}}) - \big(\max(t_{\text{hold}}) + \max(t_{\text{clock}})\big). \quad (5)$$

If the slack is negative, then the timing constraint in Formula 4 is violated. The smaller (i.e., more negative) the slack, the more critical the violation.

To reduce this formulation to a k-shortest-path graph problem, we follow standard STA practice and introduce an artificial sink node *SINK* into the circuit graph, shown as dashed edges in Figure 3. We then assign an artificial edge from each destination data point D to *SINK* with weight

$$t_{\text{D-SINK}} = -\big(\max(t_{\text{hold}}) + \max(t_{\text{clock}})\big),$$

which can be precomputed as a constant for each destination. For any timing path, the minimum accumulated delay from *CK* to *SINK* is then exactly the slack:

$$\begin{aligned} t_{\text{CK-SINK}} &= \min(t_{\text{data}}) + t_{\text{D-SINK}} \\ &= \min(t_{\text{data}}) - \big(\max(t_{\text{hold}}) + \max(t_{\text{clock}})\big) \\ &= slack. \end{aligned}$$

Consequently, the STA problem of "finding the top-k most critical timing-violating paths" can be reduced to the graph-theoretic problem of "finding the k shortest paths from *CK* to *SINK*", which can be computed in linearithmic time [9].

2.2 Common Path Pessimism Removal

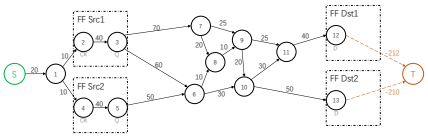
To understand how CPPR improves STA precision, let us revisit the notion of *slack* introduced in Formula 5. For the FF_2–FF_3 timing path in Figure 3, its slack is

$$\begin{aligned} slack &= \min(t_{\text{data}}) - \big(\max(t_{\text{hold}}) + \max(t_{\text{clock}})\big) \\ &= \min(t_{B_1} + t_{B_3} + t_{\text{cq}} + t_{\text{AND}}) - \max(t_{\text{hold}}) \\ &\quad - \max(t_{B_1} + t_{B_3} + t_{B_4}) \\ &= \min(t_{\text{cq}}) + \min(t_{\text{AND}}) - \max(t_{\text{hold}}) - \max(t_{B_4}) \\ &\quad - \big(\max(t_{B_1} + t_{B_3}) - \min(t_{B_1} + t_{B_3})\big). \end{aligned}$$

Observe that in this slack calculation, we *simultaneously* assumed both the maximum delay ($\max(t_{B_1} + t_{B_3})$) and minimum delay ($\min(t_{B_1} + t_{B_3})$) of the *common path* segment (B_1 and B_3), which is shared by the FF_2–FF_3 timing path and the capturing clock path. Such simultaneous assumptions are physically impossible, since the common path segment cannot exhibit both its maximum and minimum delay at the same time. Therefore, this over-conservative pessimism term should be *removed* from the slack computation—this is precisely what CPPR does. The difference between the maximum and minimum delay of the common segment is referred to as *pessimism*. In this FF_2 case, it is

$$pessimism = \max(t_{B_1} + t_{B_3}) - \min(t_{B_1} + t_{B_3}).$$

In STA with CPPR, we call the slack computed using the over-conservative Formula 5 the *pre-slack*, denoted $slack_{\text{pre}}$, and



SrcFF	DstFF	Pessimism
Src1	Dst1	5
Src2	Dst1	2
Src1	Dst2	2
Src2	Dst2	3

Figure 4. A graph representation of an example circuit. Nodes represent logic gates and flip-flops (FFs). The STA algorithm searches for paths from source FFs to destination FFs that violate timing constraints. For STA without CPPR, this is equivalent to searching for paths with the smallest weights, which correspond to pre-slack values. For STA with CPPR, pessimism must be added to rectify these weights, yielding post-slack values. The pessimism values associated with different source FF and destination FF pairs are precomputed and presented in the table on the right.

the slack after removing pessimism the *post-slack*, denoted $slack_{\text{post}}$. For any path p, their relationship is:

$$slack_{\text{post}}(p) = slack_{\text{pre}}(p) + pessimism(p). \quad (6)$$

Since $pessimism(p) \geq 0$ for any path p, we always have $slack_{\text{post}}(p) \geq slack_{\text{pre}}(p)$. That is, CPPR strictly increases slack (or leaves it unchanged), thereby reducing false timing violation reports and improving precision.

3 Motivating Example

FlashTimer's superior performance over related work stems from its critical path search strategy that reduces unnecessary path exploration to a small number. In this section, we highlight the key search strategy of FlashTimer and compare it with those of related work by examining an example circuit.

Circuit Model. Figure 4 shows the graph representation of an example circuit, where non-STA-related parts of the circuit are abstracted away. In this graph, nodes represent logic gates or flip-flops (FFs), while edges correspond to the connections between them. Each edge is assigned a delay value that indicates the time required for signals to propagate from the output of the source node to the input of the target node.

Basic Concepts. In static timing analysis with pessimism removal, the timing violation of a path p is typically assessed using a metric called *post-slack*, which quantifies the severity of the timing violation. A negative value indicates that the path violates timing constraints. This metric, denoted as $slack_{\text{post}}$, is commonly computed as

$$slack_{\text{post}}(p) = slack_{\text{pre}}(p) + pessimism(p).$$

Here, $slack_{\text{pre}}(p)$ represents the *pre-slack*, which measures the severity of a path violating timing constraints in a conservative model. On the other hand, $pessimism(p)$ refers to the overly conservative portion of $slack_{\text{pre}}(p)$.

In the example circuit shown in Figure 4, the pre-slack of a path is calculated by summing the delays (edge weights) along the path, and the pessimism values are precomputed and listed in the table on the right side of the figure. We intentionally abstract away the specific computational details for pre-slack and pessimism, as various timing models exist and our algorithm is applicable to them; understanding the exact computation is not necessary to grasp the algorithm itself. For practical implementation, benchmarking, and the examples in this work, we adopt the widely-used early-late model for pre-slack computation and apply common path pessimism removal. For comprehensive details on these and alternative timing models, we refer readers to [3, 14].

Problem to Solve. The goal in this example is to identify the k most critical timing-violating paths from source FFs to destination FFs. This is equivalent to searching for k paths with the smallest post-slack values. Note that we add a super source S and super sink T to reduce the problem to single-source, single-sink shortest paths, a common practice in graph algorithms.

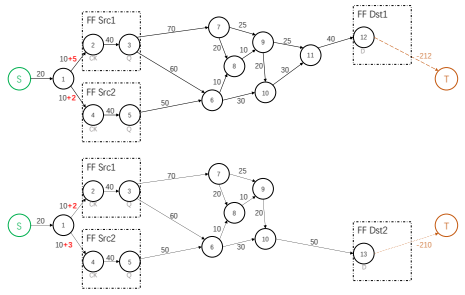
As shown in Figure 4, the two most critical timing-violating paths are:

- Path $p_1 = (S, 1, 4, 5, 6, 10, 13, T)$ with $slack_{\text{post}}(p_1) = -10 + 3 = -7$.
- Path $p_2 = (S, 1, 4, 5, 6, 8, 9, 11, 12, T)$ with $slack_{\text{post}}(p_2) = -7 + 2 = -5$.

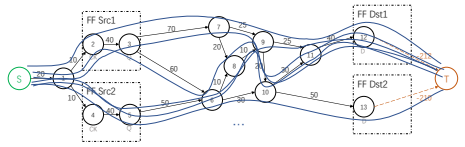
Using the search for these two paths as our running example, we demonstrate how existing algorithms categorized into two groups compare with our FlashTimer approach.

Pessimism Removal Before Search. This category of related work removes all pessimism before performing the search. OpenTimer [14] is currently the state-of-the-art open-source algorithm in this category. It requires constructing subgraphs for each destination flip-flop and performing pessimism removal on each subgraph. As shown in Figure 5a, two new subgraphs are built with edge weights rectified by pessimism so that the path length from S to T represents the post-slack. The algorithm then searches for k shortest paths on each subgraph and merges the results to find the final k critical paths. Theoretically, if the circuit has d destination FFs, the algorithm needs to initialize d subgraphs and perform kd path searches. Since d is on the order of the circuit size, excessive paths are explored when finding k critical paths, causing this algorithm to scale poorly with circuit size.

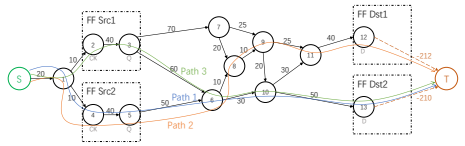
Pessimism Removal During Search. This category of related work performs pessimism removal dynamically during the path search process, with representatives such as TKTimer [18] and iTimerC [19]. Common search strategies include depth-first search (DFS) or breadth-first search (BFS), starting from the super source node, as illustrated in Figure 5b. Without optimization, these methods may need to



(a) The first category (Pessimism Removal Before Search) algorithm creates subgraphs which have done pessimism removal.



(b) The second category (Pessimism Removal During Search) algorithm uses DFS/BFS searching and prunes on the fly.



(c) FlashTimer finds first K smallest pre-slack paths efficiently, then does on query pessimism removal.

Figure 5. Illustrations of the cirtical path search strategies of different algorithms.

explore up to 12 candidate paths to identify the two worst-performing ones in the example circuit. Although various pruning techniques are employed to reduce the number of paths examined, these methods still require numerous unsuccessful trials before identifying the correct solution, which significantly impacts scalability for larger circuits.

FlashTimer. The idea behind FlashTimer is as follows: if the user requires k critical paths (i.e., the k paths with the smallest post-slack values), we first identify the $k + \hat{\alpha}$ smallest pre-slack paths using a fast shortest paths algorithm applied directly to the circuit. Next, we remove the pessimism for these paths and sort them by their post-slack values, reporting the k smallest paths.

To illustrate the process, consider the example circuit shown in Figure 5c. For this example, $\hat{\alpha} = 1$. We do not elaborate on how $\hat{\alpha}$ is calculated, as it is a technical detail. See Section 4.2 for further discussion. First, we identify three paths ($k = 2$ and $\hat{\alpha} = 1$, so $k + \hat{\alpha} = 3$): $p_1 = (S, 1, 4, 5, 6, 10, 13, T)$, $p_2 = (S, 1, 4, 5, 6, 8, 9, 11, 12, T)$, and $p_3 = (S, 1, 2, 3, 6, 10, 13, T)$, with smallest pre-slack values of -10, -7, and 0, respectively. After applying pessimism removal, the post-slack values

for these paths are calculated as follows: $slack_{\text{post}}(p_1) = -10+3 = -7$, $slack_{\text{post}}(p_2) = -7+2 = -5$, and $slack_{\text{post}}(p_3) = 0 + 2 = 2$. Thus, p_1 and p_2 are identified as the two worst paths and are reported as the final results.

The number of additional paths explored by FlashTimer (i.e., $\hat{\alpha}$) remains small because the distributions of pre-slack and post-slack values are highly correlated. In this example, the two most critical paths are exactly those with the smallest pre-slack values, demonstrating this relationship. Consequently, only a small $\hat{\alpha}$ is typically required to ensure correctness—that is, to guarantee that the true k smallest post-slack paths are included among the $k + \hat{\alpha}$ pre-slack candidates. In this example, the inclusion of just one extra path suffices to achieve correct results. This illustrates that FlashTimer can efficiently identify critical paths without excessive path exploration.

4 Algorithm

Identifying the most timing-critical paths is essential for guaranteeing circuit performance and reliability, but existing STA techniques that remove pessimism (i.e., compute post-slack) can incur prohibitive computational cost. To address this, we introduce FlashTimer, which exploits conservative but easy-to-compute *pre-slack* to guide the search for timing-violating paths with the same precision as post-slack-based methods at a fraction of the runtime. In Section 4.1, we present the overall pre-slack-guided search framework, and in Section 4.2 we detail the termination check that guarantees we capture the true top-k critical paths.

4.1 Overview

At a high level, FlashTimer employs a pre-slack-guided strategy for searching critical paths, which contrasts with related approaches that rely on post-slack to guide the search. Although some of these concepts have been introduced earlier, we now provide a more detailed explanation before introducing the searching strategy.

Critical Paths. Critical paths are paths in a circuit that violate timing constraints, and they are the expected output of Static Timing Analysis (STA) algorithms. In our discussions, we denote such paths as p or *path*.

Timing Constraints. Timing constraints are checked by STA algorithms to ensure the timing correctness of a circuit. There are two commonly used timing constraints: hold timing constraints and setup timing constraints [13, 22]. For instance, the hold timing constraint requires that a register maintains its value for a specified duration related to the clock cycle, ensuring that the register's value can propagate correctly through the circuit.

Slack. Slack is a metric computed for paths to quantify the severity of their violation of timing constraints, denoted as $slack(p)$. A negative slack value indicates a violation, and the further the slack deviates from zero, the more severe the violation. STA can be viewed as the process of identifying the k paths in a circuit with the lowest (most negative) slack values.

Pre-slack. Accurately computing slack statically is infeasible. Classic STA algorithms [13**?**] offer efficient yet conservative methods for computing slack on paths. These conservative values are referred to as *pre-slack*, denoted as $slack_{\text{pre}}(p)$, and have the following property:

$$slack_{\text{pre}}(p) \leq slack(p) \tag{7}$$

Our algorithm does not rely on a specific method for computing pre-slack; however, in our implementation, we compute it based on the mainstream early-late timing model [13**?**].

Post-slack and Pessimism. By removing overly conservative components in the computation of pre-slack, one can obtain a value known as *post-slack*, denoted as $slack_{\text{post}}(p)$. Mathematically, this is expressed as:

$$slack_{\text{post}}(p) = slack_{\text{pre}}(p) + pessimism(p). \tag{8}$$

The difference between post-slack and pre-slack is referred to as *pessimism*, which is always a non-negative value. Post-slack satisfies the following inequality:

$$slack_{\text{pre}}(p) \leq slack_{\text{post}}(p) \leq slack(p), \tag{9}$$

which shows that post-slack is a more precise representation of slack than pre-slack. Our algorithm uses the most popular common-path pessimism removal approach [13].

Based on the definitions introduced earlier, related STA methods that account for pessimism removal typically compute post-slack to guide the critical path search. However, calculating post-slack can be computationally expensive, since it requires expensive per-path pessimism computations (e.g., LCA-based common-path analysis on the clock tree) for every candidate path.

We make the key observation that allows us to avoid the computational cost of post-slack: the distributions of pre-slack and post-slack are often quite similar, as pessimism across different paths does not vary significantly. Consequently, the top-k paths with the lowest post-slack are likely to coincide with the top-$(k + \alpha)$ paths with the lowest pre-slack, where α is typically small.

Building on this observation, FlashTimer uses pre-slack to guide the path search. Specifically, FlashTimer iterates over the paths in ascending order of pre-slack, collects those with negative post-slack, and terminates when sufficient paths have been explored. This process is illustrated in Figure 6.

Intuitively, this approach achieves similar efficiency of STA algorithms that do not consider pessimism removal, as it uses pre-slack to guide the search and requires few pessimism removal. Moreover, by carefully determining the termination condition, we can achieve the same accuracy

Figure 6. Overview of the FlashTimer algorithm. FlashTimer iteratively identifies critical paths based on pre-slack, continuing the search until a sufficient solution is found.

Algorithm 1: FlashTimer

Input: circuit C, query size $k \in \mathbb{N}^+$, and tunable parameter $\beta \in \mathbb{N} \cup \{\infty\}$ for the accuracy-speed tradeoff

Output: Top k paths that may violate timing constraints in the circuit, ranked by severity. Fewer paths may be returned if no more violating paths are found.

```
// Init Phase
1  database ← initialize(C);
// Query Phase
2  Let results be an empty list;
3  foreach p in
     query_negative_preslack_paths_sorted_ascendingly(database)
   do
4      p.preslack ←
         query_preslack_for_path(database, p);
5      p.postslack ← p.preslack +
         calculate_pessimism_for_path(database, p);
6      if p.postslack ≥ 0 then
7          continue;
8      Insert p into results, maintaining ascending order
         by p.postslack;
9      if check_search_termination(results, p, k, β)
         then
10         break;
11 return the first k elements in results;
```

as algorithms that do account for pessimism removal, as discussed further in Section 4.2.

The search strategy is implemented in the pseudo-code presented in Algorithm 1. The implementation consists of two phases: initialization and query. This design ensures that repetitive initialization work is avoided when the circuit remains unchanged between user queries. We now describe these phases in more detail.

Initialization. The core operation of the initialization phase is *Database* $\leftarrow$ initialize(C), which processes the given circuit and stores relevant information in an object, *Database*, for later queries. The *Database* is primarily used to query paths with negative pre-slack in ascending order and to compute the post-slack of a path.

For querying paths with negative pre-slack, we treat this as a k-shortest paths problem, where pre-slack serves as the path weight. To support rapid pre-slack path queries, we utilize the efficient k-shortest paths algorithm from [9]. The *Database* stores the necessary information to enable this.

For computing the post-slack, as described in Equation 9, we need to calculate both pre-slack and pessimism. Pre-slack is computed in the previous step. Pessimism, on the other hand, involves finding the lowest common ancestor (LCA) of two nodes in the clock tree, since our implementation adopts the common-path pessimism removal approach [13]. We transform the original LCA problem into a Range Minimum Query (RMQ) problem [2] and employ an RMQ algorithm to efficiently compute the results online. The *Database* stores the necessary information to support this operation as well.

Query. In the query phase, FlashTimer iterates over paths with ascending negative pre-slack and collects those with negative post-slack into a list named *results* (line 3-8). The list *results* is sorted by the post-slack of the paths throughout the process. The iteration can terminate early if the predicate check_search_termination is satisfied (line 9-10), which is detailed in the next section. Once the iteration stops, the first k elements of *results* are output (line 11), representing the most critical paths that violate timing constraints.

4.2 Search Termination Check

As previously mentioned, the goal is to search for the $(k+\alpha)$ paths with the lowest pre-slack, such that the top-k paths with the lowest post-slack fall within this set. The process of determining when to terminate the path search is essentially the process of determining the value of α.

However, checking when to terminate the search in FlashTimer presents a challenge. This is because α is related to the distribution of pessimism and pre-slack, which in turn depends on the structure of the circuit. Accurately computing α is computationally expensive.

To address this, FlashTimer provides two modes to *approximate* α: **Accuracy Mode**, which over-approximates α, and **Speed Mode**, which under-approximates α. Under-approximation can result in a loss of accuracy when ranking the top-k critical paths but offers improved speed.

The mode in which FlashTimer operates is controlled by the parameter β. When $\beta = \infty$, the algorithm runs in accuracy mode. A finite value of β indicates Speed Mode, with smaller values of β corresponding to greater speed at the cost of accuracy.

Algorithm 2 presents the pseudo-code for checking search termination. We discuss how the algorithm functions in each of the two modes below.

Accuracy Mode. In accuracy mode, FlashTimer ensures that more than $k+\alpha$ paths are searched, so that the result is equivalent to searching exactly $k+\alpha$ paths. To achieve this,

Algorithm 2: check_search_termination: Search Termination Check Algorithm

Input: Path list *results* sorted by post-slack, path p newly added to *results*, query size $k \in \mathbb{N}^+$, and tunable parameter $\beta \in \mathbb{N} \cup \{\infty\}$ for the accuracy-speed tradeoff

Output: True if the path search should terminate; false otherwise

```
1 if results.size() ≥ k + β then
2     return true;
3 else if results.size() ≥ k then
4     if p.preslack > results[k − 1].postslack then
5         return true;
6 return false;
```

the termination condition is:

$$results.\text{size}() \geq k \wedge p.preslack > results[k-1].postslack. \tag{10}$$

This condition is derived from Algorithm 2 when β is set to ∞.

Next, we explain the correctness of this approach. Once we have collected k paths with negative post-slack in *results* ($results.\text{size}() \geq k$), we are unsure whether any subsequent path will have a post-slack smaller than one of the first k paths in *results*. Therefore, we must continue searching until a newly encountered path p satisfies the condition $p.preslack > results[k-1].postslack$. At this point, we can stop the search because we can conclude that any future path p' will not belong to the top k paths with the lowest post-slack. Mathematically, we can state:

$$\forall i = 0, \ldots, k, p'.postslack > results[i-1].postslack.$$

This can be proven as follows:

$$\begin{aligned} p'.postslack &\geq p'.preslack \geq p.preslack \\ &> results[k-1].postslack \\ &\geq results[i-1].postslack \ (i = 0, \ldots, k) \end{aligned}$$

The first inequality, $p'.postslack \geq p'.preslack$, follows from the property of post-slack (Inequality 9). The second inequality, $p'.preslack \geq p.preslack$, holds because we iterate paths in ascending order of their pre-slack values. The third inequality is the precondition, and the last inequality holds because *results* is sorted by post-slack.

Speed Mode. In Speed Mode, FlashTimer halts its search once it has collected $k + \beta$ negative post-slack paths, as outlined in line 1 of Algorithm 2. The parameter β is adjustable by the user. When β is small, the actual number of paths searched is fewer than $k + \alpha$, leading to an under-approximation.

This under-approximation can result in accuracy loss in ranking the paths. Specifically, the output i-th critical path may be ranked j (where $j \neq i$) in the ground truth. This occurs because the algorithm terminates early, potentially leaving some critical paths, which should have been among the top k, unexplored.

It is important to note that this under-approximation does not affect the soundness of FlashTimer, as discussed in Section 5.1.

5 Properties

In this section, we explore the key properties of FlashTimer that underpin its reliability and performance. First, we discuss its *soundness* property (Section 5.1), which ensures that users can trust FlashTimer's results: if the algorithm outputs no timing violation paths for a given circuit, the circuit is guaranteed to meet all timing constraints. Second, we examine FlashTimer's time complexity (Section 5.2) by bounding the number of paths searched (Section 5.3), which theoretically explains the innovations that enable FlashTimer's exceptional speed advantages.

5.1 Soundness

A sound STA (Static Timing Analysis) algorithm guarantees that a circuit passing the STA does not violate any timing constraints. In this section, we prove the soundness of FlashTimer.

Theorem 5.1 (Soundness)**.** *If FlashTimer outputs no paths for a given circuit, the circuit contains no critical paths.*

Proof. Assume, for the sake of contradiction, that the circuit given to FlashTimer contains m critical paths, $p_1, \ldots, p_m$, where $m > 0$. By the definition of slack and according to Inequality 9, for any $i = 1, \ldots, m$, we have:

$$slack_{\text{pre}}(p_i) \leq slack_{\text{post}}(p_i) \leq slack(p_i) < 0.$$

We now show that the *results* list is non-empty when FlashTimer returns. Let p^* denote the path among $p_1, \ldots, p_m$ with the lowest pre-slack. By the inequality above, p^* must have both negative pre-slack and post-slack. According to the FlashTimer path iteration algorithm (Algorithm 1), p^* will be the first path to be iterated over and will be added to the *results* list, from which it will never be removed. Thus, the *results* list is non-empty when FlashTimer completes.

Since the query size is positive, at least one path in the *results* list will be returned by FlashTimer, which contradicts the assumption that no paths are returned. Therefore, the soundness theorem holds. □

5.2 Time Complexity

In this section, we analyze the time complexity of FlashTimer.

Theorem 5.2 (Time Complexity)**.** *Let a circuit have n nets, and let k be the query size provided to FlashTimer. The time complexity of FlashTimer is $O(n \log n + K \log k)$, where K is the number of paths searched.*

Proof. FlashTimer operates in two phases: initialization and query.

Initialization Phase: This phase consists of three main steps:

1. Transform the given circuit into a graph representation. This operation is linear in the number of nets. Since the circuit has n nets, the transformation takes $O(n)$ time. The resulting graph has $O(n)$ nodes and $O(n)$ edges.
2. Build data structures on the graph to support path iteration sorted by pre-slack. This step involves a k-shortest paths algorithm [9] and takes $O(n \log n)$ time.
3. Build data structures for later computing post-slack values for the paths. This step involves a Lowest Common Ancestor (LCA) algorithm [2], which takes $O(n \log n)$ time.

Therefore, the total time complexity for the initialization phase is $O(n \log n)$.

Query Phase: This phase iterates over K paths, with each iteration involving the following four operations:

1. Query the next sorted negative pre-slack path: $O(\log k)$
2. Calculate the pessimism for the path: $O(\log k)$
3. Insert the path into the results and maintain the post-slack order: $O(\log k)$
4. Check the search termination condition: $O(1)$

Thus, the time complexity for the query phase is $O(K \log k)$.

In total, the time complexity of FlashTimer is $O(n \log n + K \log k)$. □

Next, we discuss how to bound K, i.e., the number of paths searched.

5.3 Bound on Path Search Complexity

In the previous section, we demonstrated that our algorithm runs in $O(n \log n + K \log k)$, where n is the circuit size, k is the number of required paths, and K is the total number of paths explored. In this section, we bound the unknown parameter K in the form of $O(k + \hat{\alpha})$ and demonstrate that $\hat{\alpha}$ can be treated as a small constant, which is crucial for the efficiency of FlashTimer.

First, we explain why the bound takes the form $O(k + \hat{\alpha})$. This form naturally arises from our pre-slack guided search strategy. Recall that to find the k critical paths, we ideally need to examine the top $k + \alpha$ paths with the smallest pre-slack values, ensuring that the k critical paths are included, where α is the minimal number of additional paths required. However, since α is difficult to determine a priori, FlashTimer approximates it with $\hat{\alpha}$, resulting in a search over K paths of the form $O(k + \hat{\alpha})$.

Next, we provide more details on the form of $\hat{\alpha}$ to demonstrate that it can be treated as a small constant. We discuss $\hat{\alpha}$ for the two modes of FlashTimer: speed mode and accuracy mode.

In speed mode, where the tunable parameter $\beta \in \mathbb{N}$, we have $K \leq k + \beta$ according to line 1 in Algorithm 2, meaning that $\hat{\alpha} = \beta$, which is typically a small constant specified by the user.

In accuracy mode, where $\beta = \infty$, the termination condition in line 4 of Algorithm 2 is more complex, making it less obvious that K is k plus a small constant. However, we show in Theorem 5.4 that K can be bounded by the maximum clock tree depth d of the given circuit, in the form

$$K \leq k + \hat{\alpha} \quad \text{where} \quad \hat{\alpha} = \left|\left\{p \mid slack_{\text{pre}}(p) \in (slack_{\text{pre}}(p_k), slack_{\text{pre}}(p_k) + d\right.\right. \tag{11}$$

Here, p_k is the path whose pre-slack ranks k-th in ascending order, and therefore $\hat{\alpha}$ represents the number of paths whose pre-slack falls within the interval $(slack_{\text{pre}}(p_k), slack_{\text{pre}}(p_k) + d]$.

Now we argue that $\hat{\alpha}$ can be treated as a small constant. The interval $(slack_{\text{pre}}(p_k), slack_{\text{pre}}(p_k) + d]$ has length d, which is typically small because the maximum clock tree depth in modern VLSI circuits is typically shallow [22] and increases slowly with circuit size. The primary concern is whether the distribution of pre-slack values is so dense that many paths fall within this small interval, making $\hat{\alpha}$ large. However, Section 6 demonstrates empirically that $\hat{\alpha}$ is generally about 10^3 in our benchmark containing circuits with millions of nets.

Since $\hat{\alpha}$ can be considered constant, we conclude that $K = O(k)$. Under this assumption, the time complexity of FlashTimer becomes:

$$O(n \log n + K \log k) = O(n \log n + k \log k). \tag{12}$$

This means FlashTimer scales nearly linearly with both the circuit size n and the number of queried paths k, which explains the efficiency of FlashTimer.

In the remainder of this section, we prove Theorem 5.4. We begin by establishing a key lemma that relates the upper bound of post-slack to pre-slack and the maximum pessimism, where the latter is bounded by the maximum depth of the clock tree. This lemma serves as the foundation for constructing $\hat{\alpha}$.

Notation. For brevity, we use more succinct notation in our proof than in previous discussions. We denote the pre-slack of a path p as $s(p)$, the post-slack as $s'(p)$, and the pessimism as $\pi(p)$. Recall that $s'(p) \triangleq s(p) + \pi(p)$, which will be used throughout the proof.

Lemma 5.3 (Post-Slack Upper Bound)**.** *Let $p_1, \ldots, p_n$ be paths in a circuit sorted by pre-slacks such that $s(p_1) \leq \cdots \leq s(p_n)$. Let $\sigma : \{1, \ldots, n\} \to \{1, \ldots, n\}$ be the permutation that sorts these paths by post-slacks in ascending order, such that $s'(p_{\sigma(1)}) \leq \cdots \leq s'(p_{\sigma(n)})$. Then for any $1 \leq i \leq n$:*

$$s'(p_{\sigma(i)}) \leq s(p_i) + \max_{1 \leq j \leq n} \pi(p_j). \tag{13}$$

Proof. Let $\pi_{\max} \triangleq \max_{1 \le j \le n} \pi(p_j)$ and define augmented pre-slacks:

$$\tilde{s}(p_i) \triangleq s(p_i) + \pi_{\max} \quad \text{for all } i \in \{1, \dots, n\}.$$

Proving this lemma is equivalent to proving that:

$$s'(p_{\sigma(i)}) \le \tilde{s}(p_i) \quad \text{for all } i \in \{1, \dots, n\}.$$

Suppose, for the sake of contradiction, that there exists some $a \in \{1, \dots, n\}$ such that:

$$s'(p_{\sigma(a)}) > \tilde{s}(p_a).$$

For any $i \in \{1, \dots, a\}$, the following chain of inequalities holds:

$$\begin{aligned} s'(p_i) &\le \tilde{s}(p_i) && \text{(since } s'(p_i) = s(p_i) + \pi(p_i) \le s(p_i) + \pi_{\max} = \tilde{s}(p_i)\text{)} \\ &\le \tilde{s}(p_a) && \text{(augmented pre-slacks preserve ordering } \tilde{s}(p_1) \le \cdots \le \tilde{s}(p_n)\text{)} \\ &< s'(p_{\sigma(a)}) && \text{(by the contradiction assumption).} \end{aligned}$$

Combining these inequalities yields $s'(p_i) < s'(p_{\sigma(a)})$ for all $i \in \{1, \dots, a\}$. This means at least a paths have post-slacks strictly less than $s'(p_{\sigma(a)})$, but by definition of σ, exactly $a - 1$ post-slacks precede $s'(p_{\sigma(a)})$ in sorted order—a contradiction.

Therefore, $s'(p_{\sigma(i)}) \le \tilde{s}(p_i) = s(p_i) + \pi_{\max}$ for all i. □

Theorem 5.4 (Path Search Complexity Bound). *Let $p_1, \dots, p_n$ be paths in a circuit sorted by pre-slacks such that $s(p_1) \le \cdots \le s(p_n)$, and let k be the query size provided to* FlashTimer. *If the maximum clock tree depth of the circuit is d, then the number of paths K searched by* FlashTimer *in accuracy mode satisfies:*

$$K \le k + |\{i \mid s(p_{k+i}) - s(p_k) \le d\}|, \tag{14}$$

which is an equivalent form of Equation 11.

Proof. According to the termination condition in accuracy mode of FlashTimer (Equation 10), the number of paths searched is given by:

$$K = \left|\left\{i \mid s(p_i) \le s'(p_{\sigma(k)})\right\}\right|.$$

Since the condition $s(p_i) \le s'(p_{\sigma(k)})$ holds for $i = 1, \dots, k$, we can express K as:

$$K = k + \left|\left\{i \mid s(p_{k+i}) \le s'(p_{\sigma(k)})\right\}\right|.$$

By Lemma 5.3, $s'(p_{\sigma(k)}) \le s(p_k) + \pi_{\max}$, where $\pi_{\max} = \max_{1 \le j \le n} \pi(p_j)$. Since the pessimism discussed in this paper is by definition bounded by the maximum clock tree depth, we have $\pi_{\max} \le d$, which gives us:

$$s'(p_{\sigma(k)}) \le s(p_k) + d.$$

Therefore:

$$\left|\left\{i \mid s(p_{k+i}) \le s'(p_{\sigma(k)})\right\}\right| \le |\{i \mid s(p_{k+i}) \le s(p_k) + d\}|.$$

Substituting this back into our expression for K:

$$K \le k + |\{i \mid s(p_{k+i}) \le s(p_k) + d\}| = k + |\{i \mid s(p_{k+i}) - s(p_k) \le d\}|.$$

□

6 Evaluation

We evaluate FlashTimer by addressing the following three research questions (RQs) in this section:

1. **Speed in Full Accuracy**: In accuracy mode, can FlashTimer achieve high speed without sacrificing accuracy?
2. **Speed-Accuracy Trade-off**: In speed mode, can FlashTimer achieve higher performance while maintaining an acceptable trade-off in accuracy?
3. **Scalability**: As the size of hardware designs and query sizes increase, can FlashTimer maintain high speed?

We answer these questions through a series of experiments, as detailed below.

Dataset. To evaluate FlashTimer, we used two datasets from the Tau Contest 2014 (Tau14) [13] and Tau Contest 2015 (Tau15) [12]. The Tau Contest is a well-regarded event sponsored by the Association for Computing Machinery (ACM), focused on the temporal aspects of digital circuits. Both the 2014 and 2015 editions emphasized static timing analysis (STA) with pessimism removal, which aligns directly with the problem FlashTimer addresses, providing a strong foundation for evaluation.

Each dataset consists of test cases that require STA algorithms to perform hold and setup timing analysis (two fundamental types of STA) on digital circuits of varying sizes, extending up to circuits with 3.8 million nets. Tables 1 summarize the key characteristics of the test cases in the Tau14 and Tau15 datasets, respectively, reflecting the scale of the circuits involved.

Additionally, each test case is associated with a query size k, which requires the STA algorithm to output the top-k most critical paths in the circuit that violate timing constraints. For the Tau14 dataset, we used the officially designated query sizes for each test case. However, the Tau15 dataset does not provide official query size recommendations, so we selected query sizes of 1000, 10000, and 100000 for each test case. These values were chosen based on a prior study [11], and they also serve to assess the scalability of FlashTimer with respect to query size.

Note that the Tau14 dataset is available in two versions: a simpler version and a more complex one. We chose the more complex version, as it features a higher proportion of challenging clock trees, which pose greater challenges for our algorithm.

Baseline. For comparison, we selected two open-source algorithms as baselines, both of which are well-regarded in this domain: *OpenTimer* [15] and *TKTimer* [18]. Due to input/output format compatibility issues between the datasets, TKTimer supports only the Tau14 dataset, while OpenTimer supports only the Tau15 dataset. To ensure a comprehensive evaluation, FlashTimer was implemented to work with both datasets. All algorithms were implemented in C++ to ensure fairness in the comparison.

Table 1. Specifications of Tau14 and Tau15 datasets. The columns "Pis" and "Pos" represent the number of primary inputs and primary outputs, respectively, which represent the external inputs and outputs of the circuit module. "Nets" indicates the number of nets in the circuit (with "K" representing 10^3). The number of nets serves as a rough indicator of the scale of the circuit's design.

	Index	Circuit Name	Pis	Pos	Nets	Index	Circuit Name	Pis	Pos	Nets
Tau 14	1	ac97_ctrl	84	48	55.7K	10	vga_lcd	89	109	525.6K
	2	aes_core	260	129	86.7K	11	wb_dma	217	215	17.4K
	3	des_perf	235	64	404.2K	12	Combo2	170	218	284.4K
	4	mem_ctrl	115	152	45.0K	13	Combo3	353	215	216.2K
	5	pci_bridge32	162	207	78.2K	14	Combo4	260	169	866.3K
	6	systemcaes	260	129	29.6K	15	Combo5	432	164	2229.6K
	7	systemcdes	132	65	13.3K	16	Combo6	486	174	3843.9K
	8	tv80	14	32	23.7K	17	Combo7	459	148	3012.3K
	9	usb_funct	128	121	66.1K					
Tau 15	18	crc32d16N	19	32	495	32	wb_dma	217	215	4.4K
	19	mgc_matrix_mult_iccad	3.2K	1.6K	174.5K	33	leon3mp_iccad	254	79	1248.0K
	20	cordic_ispd	34	64	45.4K	34	mem_ctrl	115	152	10.7K
	21	leon2_iccad	615	85	1517.0K	35	usb_phy_ispd	15	19	938
	22	tip_master	778	869	38.5K	36	aes_core	260	129	23.2K
	23	softusb_navre	34	51	7.0K	37	netcard_iccad	1.8K	10	1497.8K
	24	pci_bridge32_ispd	160	201	41.0K	38	usb_funct	128	121	15.9K
	25	mgc_edit_dist_iccad	2.6K	12	164.2K	39	tv80	14	32	5.3K
	26	systemcdes	132	65	3.6K	40	systemcaes	260	129	6.8K
	27	ac97_ctrl	84	48	14.4K	41	fft_ispd	1.0K	2.0K	39.2K
	28	vga_lcd_iccad	85	99	259.1K	42	des_perf_ispd	234	140	139.1K
	29	matrix_mult_ispd	3.2K	1.6K	167.2K	43	vga_lcd	89	109	139.6K
	30	cordic_dut	80	78	3.6K	44	des_perf	235	64	106.5K
	31	edit_dist_ispd	2.6K	12	150.2K	45	b19_iccad	22	25	255.3K

Metrics. We evaluate STA algorithms using two metrics: *accuracy* and *speed*.

- *Accuracy* measures how well the k queried critical paths produced by the algorithm align with the ground truth provided by the dataset. Specifically, accuracy is defined as:

$$accuracy = \frac{1}{k}\sum_{i=1}^{k} \mathbb{I}\left(\text{the } i\text{-th path in output} = \text{the } i\text{-th path in ground truth}\right) \tag{15}$$

  where $\mathbb{I}(\cdot)$ is the indicator function. The accuracy value ranges from 0 to 1, with a value closer to 1 indicating better alignment with the ground truth, meaning the algorithm has ranked the top-k critical paths more accurately.
- *Speed* is measured by the runtime of the algorithm, excluding I/O time. I/O time can vary depending on the implementation of I/O operations and is not central to the evaluation of the STA algorithm itself. To reduce the impact of system fluctuations, each experiment was repeated at least four times, and the average runtime is reported.

Algorithm Configuration. To address RQ1 and RQ2, we conducted experiments with FlashTimer across different accuracy-speed trade-offs by tuning its parameter β to the values ∞, 1000, and 10. When β is set to ∞, FlashTimer operates in accuracy mode. When β is set to 1000 or 10, it operates in speed mode, with smaller values of β resulting in lower accuracy but increased speed. We exclude results for $\beta > 1000$ since $\beta = 1000$ already achieves near-optimal accuracy. Results for $\beta = 100$ and other intermediate values are omitted due to space limitations, and $\beta = 10$ is already representative of the speed-accuracy trade-off.

Experimental Environment. All experiments were conducted on a machine equipped with 128GB of RAM and an Intel Xeon(R) Silver 4114 processor.

Results. Tables 2, 3 and 4 summarize all the experimental results, which are discussed in detail in the following sections. Note that for the Tau14 dataset, the results for hold and setup

timing analysis are reported separately. For the Tau15 dataset, these results are combined, meaning paths that violate either hold or setup timing constraints are ranked and reported together, as required by the dataset.

6.1 RQ1: Speed in Full Accuracy

High-speed STA algorithms can dramatically shorten iteration cycles in the verification of large-scale chips, making runtime efficiency the core design objective of FlashTimer. In this section, we quantify the speedup achieved by FlashTimer over baseline algorithms in its accuracy mode. The experimental results are summarized in the "**Query Part** ($\beta = \infty$)" columns of Tables 2, 3, and 4.

In the Tau14 dataset, FlashTimer achieves an average speedup of 48.40 times, with a maximum speedup of 559.85 times. In the Tau15 dataset, the average speedup is 70.81 times, with a maximum of 837.01 times. In both datasets, the accuracy remains 100%, as expected.

However, when analyzing small-scale circuits with small-scale queries, such as the circuits indexed 18 and 19, FlashTimer does not show a performance advantage. This is because FlashTimer performs extensive preprocessing in the initialization phase to accelerate subsequent searches. When both the design and query sizes are sufficiently small, the preprocessing time does not outweigh the acceleration of the subsequent searches. As a result, FlashTimer may underperform compared to other search strategies used by the baseline.

Additionally, in some circuits with small-scale designs (e.g., indexed 18) but larger query sizes (e.g., 100,000), FlashTimer still fails to provide a speedup. This is because the total number of paths in the circuit is smaller than the large query size, making the large query size effectively no different from a small query size.

Notably, the speedup of FlashTimer grows with the query size. This is because its query cost is approximately $O(k \log k)$ (see discussion in Section 5.3), which rises far more slowly with k than those of the baseline methods; thus, larger queries amplify its advantage.

> ***Conclusion.*** FlashTimer can achieve significant speedups without sacrificing accuracy. On average, the algorithm provides a speedup of 59.61 times over the baselines across both datasets in our benchmark.

6.2 RQ2: Speed-Accuracy Trade-off

FlashTimer offers a speed mode designed to sacrifice some accuracy in exchange for even greater speed. In this section, we examine the speedup gained by sacrificing accuracy.

As mentioned earlier, the algorithm parameter β controls the accuracy of FlashTimer: smaller values of β result in greater accuracy sacrifice. The columns labeled "**Query Part** ($\beta = 1000$)" and "**Query Part** ($\beta = 10$)" in Tables 2, 3 and 4 show the speedup and accuracy of FlashTimer in the Tau14 and Tau15 datasets for different values of β.

When $\beta = 1000$, there is a slight sacrifice in accuracy, with an average loss of 0.40% across both datasets. In exchange, the average speedup relative to the baseline is 72.81 times, with a maximum speedup of 837.01 times.

For comparison, in accuracy mode, the speedup is 59.61 times across both datasets. This result indicates that in accuracy mode, the number of paths explored is only slightly higher than that in speed mode with $\beta = 1000$, which is approximately $k + 10^3$, thus explaining why accuracy mode also achieves high speed.

When $\beta = 10$, the accuracy is sacrificed more, with an average loss of 1.40% across both datasets, but the accuracy remains within an acceptable range. With this greater sacrifice, FlashTimer achieves an average speedup of 113.17 times, with a maximum speedup of 1773.03 times.

> ***Conclusion.*** A small sacrifice in accuracy when ranking the top critical paths can lead to significant speedups. In our benchmark, sacrificing 1–2% of accuracy results in an average speedup of 113.17 times. This experiment also empirically confirms our theoretical analysis: to maintain full precision, the extra number of paths explored remains modest—on the order of 10^3.

6.3 RQ3: Scalability

Modern large digital circuits present significant scalability challenges for static timing analysis algorithms, which previous STA algorithms with CPPR have failed to address effectively. In this section, we evaluate the scalability of FlashTimer with respect to its two main inputs: circuit size and query size. Theoretical analysis in Section 5.3 shows that the time complexity of FlashTimer is $O(n \log n + k \log k)$, where n is the circuit size and k is the query size. Thus, we expect FlashTimer to scale nearly linearly with both circuit size and query size.

The running time of FlashTimer across various circuit and query sizes is already presented in Tables 2, 3, and 4. The circuits range from 495 to 3.84 million nets, with the number of nets serving as an indicator of circuit size. Query sizes range from 1,000 to 700,000, covering commonly used values. Based on these data, we discuss scalability with respect to circuit size and query size separately in the following paragraphs.

Circuit Size Scalability. To provide a clearer view of circuit size scalability based on data from Tables 2, 3 and 4, we plot the trend of FlashTimer's initialization time (denoted as Init Part Time) with respect to the number of nets, which indicates circuit size. We exclude the query part time

Table 2. Performance data for FlashTimer and TKTimer across multiple cases on Tau14 datasets. The Case Index corresponds to Table 1. The Init Part represents the time statistics of the initialization phase of FlashTimer, which is solely dependent on the design itself and independent of the query size. The hold and setup in the Type section represent the two basic analyses in timing analysis. The Query Size, denoted as k, refers to the number of critical paths in the query requirements. The Query Part shows the statistics for the query phase of FlashTimer, with tests conducted under different values of α. The Time column includes the time, with the value in parentheses indicating the speedup ratio of the total FlashTimer time (the sum of Init Part and Query Part times) relative to TKTimer under the corresponding condition. Accuracy displays the precision of the results, as defined in the Metrics 6 section. The TKTimer column presents the performance of TKTimer.

Case Index	Query Size k	Type	TKTimer		Init Part	Query Part ($\alpha = \infty$)		Query Part ($\alpha = 1000$)		Query Part ($\alpha = 10$)	
			Time (s)	Accuracy	Time (s)	Time (s)	Accuracy	Time (s)	Accuracy	Time (s)	Accuracy
1	1000	hold	0.071	100.00%	0.078	0.017 (0.75x)	100.00%	0.018 (0.74x)	100.00%	0.014 (0.77x)	100.00%
		setup	0.054	100.00%		0.010 (0.61x)	100.00%	0.006 (0.64x)	100.00%	0.008 (0.63x)	99.50%
2	1000	hold	0.427	100.00%	0.106	0.054 (2.67x)	100.00%	0.065 (2.50x)	100.00%	0.008 (3.75x)	99.50%
		setup	0.456	100.00%		0.094 (2.28x)	100.00%	0.134 (1.90x)	100.00%	0.010 (3.93x)	98.50%
3	1000	hold	1.433	100.00%	1.047	0.070 (1.28x)	100.00%	0.074 (1.28x)	100.00%	0.034 (1.33x)	100.00%
		setup	1.412	100.00%		0.069 (1.27x)	100.00%	0.075 (1.26x)	100.00%	0.032 (1.31x)	100.00%
4	1000	hold	0.252	100.00%	0.049	0.027 (3.32x)	100.00%	0.042 (2.77x)	100.00%	0.010 (4.27x)	100.00%
		setup	0.283	100.00%		0.060 (2.60x)	100.00%	0.071 (2.36x)	100.00%	0.012 (4.64x)	99.50%
5	1000	hold	0.815	100.00%	0.127	0.028 (5.26x)	100.00%	0.035 (5.03x)	100.00%	0.024 (5.40x)	100.00%
		setup	0.910	100.00%		0.098 (4.04x)	100.00%	0.130 (3.54x)	100.00%	0.028 (5.87x)	99.50%
6	1000	hold	49.827	100.00%	0.029	0.060 (559.85x)	100.00%	0.098 (392.34x)	100.00%	0.010 (1277.62x)	100.00%
		setup	62.721	100.00%		3.505 (17.75x)	100.00%	0.240 (233.16x)	100.00%	0.012 (1529.78x)	98.00%
7	1000	hold	0.236	100.00%	0.011	0.151 (1.46x)	100.00%	0.048 (4.00x)	100.00%	0.004 (15.73x)	100.00%
		setup	0.298	100.00%		0.195 (1.45x)	100.00%	0.068 (3.77x)	99.46%	0.004 (19.87x)	99.46%
8	1000	hold	41.810	100.00%	0.02	0.112 (316.74x)	100.00%	0.056 (550.13x)	100.00%	0.008 (1493.21x)	100.00%
		setup	51.418	100.00%		1.198 (42.22x)	100.00%	0.151 (300.69x)	100.00%	0.009 (1773.03x)	98.50%
9	1000	hold	0.352	100.00%	0.085	0.027 (3.14x)	100.00%	0.038 (2.86x)	100.00%	0.014 (3.56x)	100.00%
		setup	0.368	100.00%		0.069 (2.39x)	100.00%	0.067 (2.42x)	100.00%	0.015 (3.68x)	100.00%
10	1000	hold	4.474	100.00%	1.52	0.106 (2.75x)	100.00%	0.114 (2.74x)	100.00%	0.096 (2.77x)	100.00%
		setup	4.588	100.00%		0.099 (2.83x)	100.00%	0.085 (2.86x)	100.00%	0.075 (2.88x)	99.00%
11	1000	hold	0.031	100.00%	0.015	0.006 (1.48x)	100.00%	0.009 (1.29x)	100.00%	0.005 (1.55x)	100.00%
		setup	0.033	100.00%		0.008 (1.43x)	100.00%	0.009 (1.38x)	100.00%	0.006 (1.57x)	100.00%
12	150000	hold	3.952	100.00%	0.859	0.112 (4.07x)	100.00%	0.099 (4.13x)	100.00%	0.067 (4.27x)	99.99%
		setup	4.453	100.00%		0.225 (4.11x)	100.00%	0.228 (4.10x)	99.98%	0.107 (4.61x)	99.98%
13	120000	hold	1.786	100.00%	0.507	0.096 (2.96x)	100.00%	0.086 (3.01x)	100.00%	0.048 (3.22x)	100.00%
		setup	2.091	100.00%		0.129 (3.29x)	100.00%	0.121 (3.33x)	99.91%	0.061 (3.68x)	99.91%
14	225000	hold	364.934	100.00%	3.142	0.534 (99.27x)	100.00%	0.538 (99.17x)	100.00%	0.204 (109.07x)	100.00%
		setup	509.937	100.00%		53.035 (9.08x)	100.00%	0.774 (130.22x)	99.99%	0.236 (150.96x)	99.25%
15	300000	hold	1295.410	100.00%	9.685	4.077 (94.13x)	100.00%	0.825 (123.25x)	100.00%	0.279 (130.01x)	99.97%
		setup	1400.190	100.00%		20.908 (45.77x)	100.00%	1.465 (125.58x)	99.99%	0.319 (139.96x)	98.66%
16	525000	hold	1368.000	100.00%	18.234	0.888 (71.54x)	100.00%	5.980 (56.50x)	100.00%	0.470 (73.14x)	99.96%
		setup	1442.400	100.00%		8.898 (53.16x)	100.00%	5.901 (59.76x)	99.99%	0.521 (76.91x)	99.93%
17	700000	hold	3301.210	100.00%	13.832	1.690 (212.68x)	100.00%	1.335 (217.66x)	100.00%	0.548 (229.57x)	99.96%
		setup	3525.150	100.00%		38.026 (67.98x)	100.00%	2.709 (213.12x)	99.97%	0.544 (245.21x)	99.92%

of FlashTimer from consideration, as it is not affected by circuit size, according to our complexity analysis (Theorem 5.2).

The plotted charts for the Tau14 and Tau15 datasets are shown in Figures 7. From these figures, it is evident that the initialization time grows approximately linearly with the number of nets, which is consistent with our theoretical analysis of $O(n \log n)$ complexity.

Query Size Scalability. From Tables 3 and 4, we observe that FlashTimer's runtime increases approximately linearly with query size. To further investigate this behavior on large hardware designs and to compare with OpenTimer, we conducted a case study on the `leon2_iccad` and `netcard_iccad` circuits (indexed 21 and 37), sampling additional query sizes in the range $[10^3, 10^6]$ to analyze the runtime trends of FlashTimer. These two circuits were selected because they

Table 3. Performance Comparison of FlashTimer and OpenTimer across multiple cases on Tau15 datasets (Part 1). The Case Index corresponds to Table 1. The Init Part represents the time statistics of the initialization phase of FlashTimer, which is solely dependent on the design itself and independent of the query volume. The Query Size, denoted as K, refers to the number of key paths in the query requirements. The Query Part shows the statistics for the query phase of FlashTimer, with tests conducted under different values of α. The Time column includes the time, with the value in parentheses indicating the speedup ratio of the total FlashTimer time (the sum of Init Part and Query Part times) relative to OpenTimer under the corresponding condition. Accuracy displays the precision of the results, as defined in the Metrics 6 section. The OpenTimer column presents the performance of OpenTimer.

Case	Query Size	OpenTimer		Init Part	Query Part ($\alpha = \infty$)		Query Part ($\alpha = 1000$)		Query Part ($\alpha = 10$)	
Index	K	Time (s)	Accuracy	Time (s)	Time (s)	Accuracy	Time (s)	Accuracy	Time (s)	Accuracy
18	1000	0.004	100.00%	0.019	0.004 (0.17x)	100.00%	0.004 (0.17x)	100.00%	0.004 (0.17x)	100.00%
	10000	0.003	100.00%		0.005 (0.12x)	100.00%	0.005 (0.12x)	100.00%	0.006 (0.12x)	100.00%
	100000	0.003	100.00%		0.005 (0.12x)	100.00%	0.005 (0.12x)	100.00%	0.005 (0.12x)	100.00%
19	1000	0.896	100.00%	5.47	0.009 (0.16x)	100.00%	0.01 (0.16x)	100.00%	0.01 (0.16x)	100.00%
	10000	117.979	100.00%		0.014 (21.51x)	100.00%	0.012 (21.52x)	100.00%	0.013 (21.52x)	100.00%
	100000	1531.9	100.00%		0.077 (276.17x)	100.00%	0.085 (275.77x)	100.00%	0.075 (276.27x)	100.00%
20	1000	5.983	100.00%	1.563	0.006 (3.81x)	100.00%	0.005 (3.82x)	100.00%	0.004 (3.82x)	100.00%
	10000	130.78	100.00%		0.019 (82.67x)	100.00%	0.019 (82.67x)	99.99%	0.018 (82.72x)	99.99%
	100000	1386.17	100.00%		0.129 (819.25x)	100.00%	0.099 (834.04x)	100.00%	0.099 (834.04x)	100.00%
21	1000	23.258	100.00%	76.719	0.005 (0.30x)	100.00%	0.004 (0.30x)	100.00%	0.006 (0.30x)	100.00%
	10000	678.381	100.00%		0.014 (8.84x)	100.00%	0.011 (8.84x)	100.00%	0.015 (8.84x)	100.00%
	100000	19507.6	100.00%		0.069 (254.04x)	100.00%	0.074 (254.03x)	100.00%	0.082 (254.00x)	99.92%
22	1000	0.484	100.00%	1.484	0.004 (0.33x)	100.00%	0.005 (0.33x)	100.00%	0.005 (0.33x)	100.00%
	10000	28.363	100.00%		0.015 (18.92x)	100.00%	0.014 (18.93x)	100.00%	0.014 (18.93x)	99.95%
	100000	441.232	100.00%		0.076 (282.84x)	100.00%	0.062 (285.40x)	100.00%	0.069 (284.12x)	99.58%
23	1000	1.647	100.00%	0.21	0.005 (7.66x)	100.00%	0.005 (7.66x)	100.00%	0.005 (7.66x)	96.00%
	10000	13.345	100.00%		0.013 (59.84x)	100.00%	0.013 (59.84x)	100.00%	0.013 (59.84x)	96.34%
	100000	91.695	100.00%		0.068 (329.84x)	100.00%	0.063 (335.88x)	97.89%	0.064 (334.65x)	96.96%
24	1000	0.365	100.00%	1.314	0.011 (0.28x)	100.00%	0.004 (0.28x)	100.00%	0.005 (0.28x)	100.00%
	10000	8.487	100.00%		0.016 (6.38x)	100.00%	0.021 (6.36x)	100.00%	0.016 (6.38x)	100.00%
	100000	61.453	100.00%		0.085 (43.93x)	100.00%	0.074 (44.27x)	100.00%	0.072 (44.34x)	99.79%
25	1000	18.744	100.00%	8.165	0.006 (2.29x)	100.00%	0.005 (2.29x)	100.00%	0.004 (2.29x)	100.00%
	10000	318.931	100.00%		0.015 (38.99x)	100.00%	0.014 (38.99x)	100.00%	0.013 (39.00x)	98.66%
	100000	1550.65	100.00%		0.071 (188.28x)	100.00%	0.065 (188.41x)	98.91%	0.067 (188.37x)	97.96%
26	1000	0.225	100.00%	0.097	0.005 (2.21x)	100.00%	0.005 (2.21x)	100.00%	0.004 (2.23x)	95.00%
	10000	2.276	100.00%		0.012 (20.88x)	100.00%	0.012 (20.88x)	100.00%	0.012 (20.88x)	98.31%
	100000	7.76	100.00%		0.101 (39.19x)	100.00%	0.076 (44.86x)	100.00%	0.076 (44.86x)	99.37%
27	1000	0.079	100.00%	0.632	0.004 (0.12x)	100.00%	0.003 (0.12x)	100.00%	0.004 (0.12x)	99.50%
	10000	1.928	100.00%		0.016 (2.98x)	100.00%	0.016 (2.98x)	100.00%	0.015 (2.98x)	96.93%
	100000	5.029	100.00%		0.084 (7.02x)	100.00%	0.071 (7.15x)	100.00%	0.072 (7.14x)	100.00%
28	1000	0.227	100.00%	10.176	0.007 (0.02x)	100.00%	0.014 (0.02x)	100.00%	0.011 (0.02x)	99.30%
	10000	5.63	100.00%		0.034 (0.55x)	100.00%	0.019 (0.55x)	100.00%	0.019 (0.55x)	99.68%
	100000	262.587	100.00%		0.092 (25.57x)	100.00%	0.081 (25.60x)	100.00%	0.078 (25.61x)	99.71%
29	1000	3.256	100.00%	7.086	0.006 (0.46x)	100.00%	0.006 (0.46x)	100.00%	0.006 (0.46x)	100.00%
	10000	173.73	100.00%		0.014 (24.47x)	100.00%	0.02 (24.45x)	99.99%	0.016 (24.46x)	99.99%
	100000	2258.63	100.00%		0.09 (314.75x)	100.00%	0.084 (315.01x)	99.99%	0.084 (315.01x)	99.99%
30	1000	1.553	100.00%	0.085	0.004 (17.45x)	100.00%	0.004 (17.45x)	100.00%	0.004 (17.45x)	100.00%
	10000	12.658	100.00%		0.013 (129.16x)	100.00%	0.013 (129.16x)	100.00%	0.013 (129.16x)	99.35%
	100000	37.782	100.00%		0.082 (226.24x)	100.00%	0.062 (257.02x)	99.78%	0.065 (251.88x)	98.78%
31	1000	48.443	100.00%	5.443	0.005 (8.89x)	100.00%	0.004 (8.89x)	100.00%	0.005 (8.89x)	100.00%
	10000	458.095	100.00%		0.016 (83.92x)	100.00%	0.021 (83.84x)	100.00%	0.015 (83.93x)	98.88%
	100000	4632.03	100.00%		0.091 (837.01x)	100.00%	0.091 (837.01x)	100.00%	0.098 (835.96x)	98.98%

are the largest designs that can be run with OpenTimer. TKTimer was not included in this comparison due to the greater popularity of OpenTimer.

Figure 8 illustrates the relationship between query size and runtime for both FlashTimer and OpenTimer on a logarithmic scale. It is evident that FlashTimer exhibits excellent scalability, with query size variations having minimal impact on total runtime. In contrast, OpenTimer demonstrates a quadratic complexity disadvantage, with runtime increasing significantly for large-scale queries.

Table 4. Performance Comparison of FlashTimer and OpenTimer across multiple cases on Tau15 datasets (Part 2, continued part of Table 3).

Case Index	Query Size K	OpenTimer Time (s)	OpenTimer Accuracy	Init Part Time (s)	Query Part ($\alpha = \infty$) Time (s)	Query Part ($\alpha = \infty$) Accuracy	Query Part ($\alpha = 1000$) Time (s)	Query Part ($\alpha = 1000$) Accuracy	Query Part ($\alpha = 10$) Time (s)	Query Part ($\alpha = 10$) Accuracy
32	1000	0.072	100.00%	0.12	0.005 (0.58x)	100.00%	0.005 (0.58x)	100.00%	0.004 (0.58x)	98.10%
	10000	0.674	100.00%		0.013 (5.07x)	100.00%	0.014 (5.03x)	100.00%	0.013 (5.07x)	97.32%
	100000	0.727	100.00%		0.056 (4.13x)	100.00%	0.053 (4.20x)	100.00%	0.055 (4.15x)	100.00%
33	1000	8.403	100.00%	55.324	0.006 (0.15x)	100.00%	0.005 (0.15x)	100.00%	0.006 (0.15x)	100.00%
	10000	333.191	100.00%		0.013 (6.02x)	100.00%	0.014 (6.02x)	100.00%	0.011 (6.02x)	100.00%
	100000	6295.98	100.00%		0.069 (113.66x)	100.00%	0.073 (113.65x)	100.00%	0.07 (113.66x)	100.00%
34	1000	0.507	100.00%	0.489	0.005 (1.03x)	100.00%	0.005 (1.03x)	100.00%	0.004 (1.03x)	99.10%
	10000	4.578	100.00%		0.02 (8.99x)	100.00%	0.017 (9.05x)	98.75%	0.019 (9.01x)	89.86%
	100000	29.444	100.00%		0.098 (50.16x)	100.00%	0.093 (50.59x)	95.35%	0.082 (51.57x)	94.35%
35	1000	0.01	100.00%	0.032	0.004 (0.28x)	100.00%	0.004 (0.28x)	100.00%	0.004 (0.28x)	100.00%
	10000	0.008	100.00%		0.006 (0.21x)	100.00%	0.006 (0.21x)	100.00%	0.007 (0.21x)	100.00%
	100000	0.007	100.00%		0.005 (0.19x)	100.00%	0.005 (0.19x)	100.00%	0.005 (0.19x)	100.00%
36	1000	0.989	100.00%	0.732	0.008 (1.34x)	100.00%	0.005 (1.34x)	100.00%	0.004 (1.34x)	96.30%
	10000	13.006	100.00%		0.018 (17.34x)	100.00%	0.017 (17.36x)	100.00%	0.018 (17.34x)	97.36%
	100000	28.8	100.00%		0.099 (34.66x)	100.00%	0.087 (35.16x)	99.97%	0.095 (34.82x)	98.98%
37	1000	5.301	100.00%	62.702	0.011 (0.08x)	100.00%	0.011 (0.08x)	100.00%	0.008 (0.08x)	100.00%
	10000	194.942	100.00%		0.016 (3.11x)	100.00%	0.014 (3.11x)	100.00%	0.02 (3.11x)	99.90%
	100000	1084.38	100.00%		0.08 (17.27x)	100.00%	0.08 (17.27x)	100.00%	0.086 (17.27x)	99.64%
38	1000	0.298	100.00%	0.702	0.004 (0.42x)	100.00%	0.005 (0.42x)	100.00%	0.004 (0.42x)	98.70%
	10000	3.314	100.00%		0.016 (4.62x)	100.00%	0.017 (4.61x)	99.88%	0.015 (4.62x)	98.72%
	100000	32.126	100.00%		0.087 (40.72x)	100.00%	0.071 (41.56x)	97.44%	0.069 (41.67x)	96.43%
39	1000	1.145	100.00%	0.181	0.006 (6.12x)	100.00%	0.006 (6.12x)	100.00%	0.005 (6.16x)	83.60%
	10000	12.204	100.00%		0.024 (59.53x)	100.00%	0.021 (60.42x)	87.42%	0.019 (61.02x)	80.27%
	100000	90.036	100.00%		0.117 (302.13x)	100.00%	0.09 (332.24x)	86.32%	0.093 (328.60x)	85.39%
40	1000	0.842	100.00%	0.344	0.006 (2.41x)	100.00%	0.006 (2.41x)	100.00%	0.004 (2.42x)	85.70%
	10000	21.827	100.00%		0.02 (59.96x)	100.00%	0.02 (59.96x)	94.79%	0.017 (60.46x)	86.90%
	100000	230.125	100.00%		0.098 (520.64x)	100.00%	0.104 (513.67x)	96.80%	0.101 (517.13x)	95.86%
41	1000	0.415	100.00%	0.95	0.008 (0.43x)	100.00%	0.007 (0.43x)	100.00%	0.003 (0.44x)	100.00%
	10000	29.771	100.00%		0.014 (30.88x)	100.00%	0.017 (30.79x)	100.00%	0.017 (30.79x)	100.00%
	100000	278.478	100.00%		0.061 (275.45x)	100.00%	0.066 (274.09x)	100.00%	0.065 (274.36x)	100.00%
42	1000	1.144	100.00%	5.418	0.004 (0.21x)	100.00%	0.006 (0.21x)	100.00%	0.003 (0.21x)	100.00%
	10000	10.037	100.00%		0.018 (1.85x)	100.00%	0.018 (1.85x)	100.00%	0.012 (1.85x)	100.00%
	100000	104.304	100.00%		0.078 (18.98x)	100.00%	0.073 (19.00x)	100.00%	0.073 (19.00x)	100.00%
43	1000	0.576	100.00%	5.598	0.008 (0.10x)	100.00%	0.008 (0.10x)	100.00%	0.009 (0.10x)	99.70%
	10000	7.556	100.00%		0.027 (1.34x)	100.00%	0.028 (1.34x)	100.00%	0.027 (1.34x)	99.30%
	100000	395.895	100.00%		0.095 (69.54x)	100.00%	0.097 (69.52x)	99.98%	0.088 (69.63x)	99.01%
44	1000	1.04	100.00%	4.474	0.008 (0.23x)	100.00%	0.004 (0.23x)	100.00%	0.006 (0.23x)	100.00%
	10000	7.995	100.00%		0.019 (1.78x)	100.00%	0.023 (1.78x)	100.00%	0.023 (1.78x)	100.00%
	100000	138.362	100.00%		0.112 (30.17x)	100.00%	0.111 (30.18x)	100.00%	0.119 (30.12x)	100.00%
45	1000	2.764	100.00%	15.642	0.007 (0.18x)	100.00%	0.007 (0.18x)	100.00%	0.007 (0.18x)	96.60%
	10000	90.006	100.00%		0.02 (5.75x)	100.00%	0.02 (5.75x)	99.99%	0.016 (5.75x)	99.61%
	100000	984.469	100.00%		0.106 (62.51x)	100.00%	0.1 (62.54x)	100.00%	0.103 (62.53x)	99.97%

Conclusion. FlashTimer demonstrates excellent scalability for both circuit size and query size, with runtime increasing approximately linearly in both cases. This makes FlashTimer well-suited for analyzing modern large-scale digital circuits.

7 Related Work

Common-path pessimism removal (CPPR) is essential for reducing false positives in static timing analysis (STA) of modern VLSI designs [22]. However, existing STA algorithms with CPPR can incur a 10–100× runtime penalty on large circuits. Our FlashTimer approach significantly reduces this overhead by restricting expensive computations to approximately the most promising candidate paths, while maintaining soundness. This section briefly reviews related STA algorithms with CPPR, then summarizes FlashTimer's advantages relative to these methods, and finally discusses the timing models referenced but not detailed in Section 3.

STA with CPPR. In 2014, the CPPR challenge was formally introduced at the TAU Contest 2014 (Tau14) [13], spurring the first wave of specialized STA algorithms; representative systems from that year include iTimerC [19, 23], TkTimer [18], and UI-Timer [16]. iTimerC and TkTimer

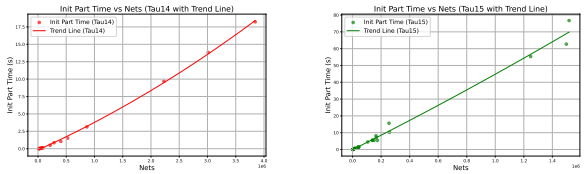


Figure 7. Scalability of FlashTimer with respect to circuit size, as indicated by the trend of initialization time with respect to the number of nets in the circuit. Both trends exhibit approximately linear growth.

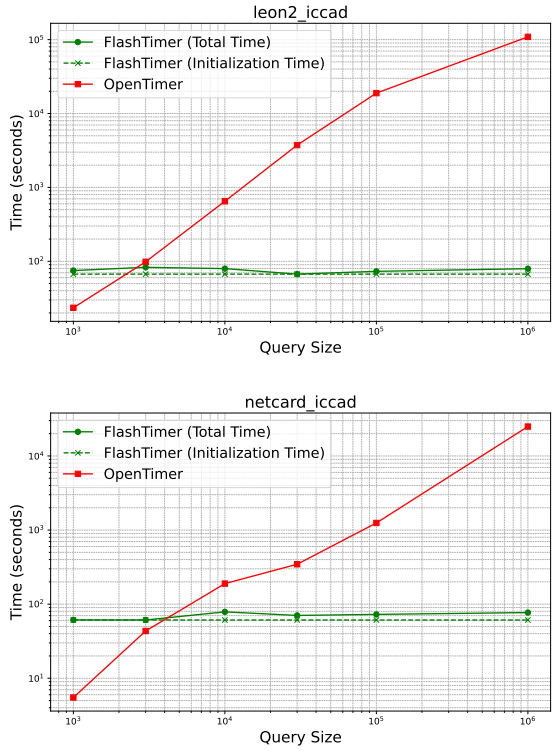


Figure 8. Scalability of FlashTimer with respect to query size, compared with OpenTimer. FlashTimer's performance is less affected by query size compared to OpenTimer.

both follow a pessimism-removal-during-search paradigm to prune the path-generation search space, and perform well when the topological alternatives between flip-flop pairs are limited—in other words, when the number of distinct paths carrying a signal from a source flip-flop to a destination flip-flop is often fewer than 100. In the same contest, UI-Timer (paper published in 2016) won Tau14 and introduced the key idea of performing pessimism removal on a per-destination-flip-flop basis before running a k-shortest-paths search on the reduced graph, substantially improving scalability. For our Tau14-era baseline, we adopt *TkTimer* rather than UI-Timer, because UI-Timer (a before-search CPPR realization) was later realized within OpenTimer by the same authors; since we also select OpenTimer as a baseline, UI-Timer is implicitly covered by our comparisons. We do not select other Tau14-era systems (e.g., iTimerC) as baselines because they do not provide public code; we attempted to contact the respective authors but were unable to obtain implementations or permission to use internal versions. Moreover, their runtime degrades markedly as instance sizes grow and, in most cases, they underperform OpenTimer (UI-Timer).

In 2015, the TAU Contest 2015 [12] continued to build on STA with CPPR. That year's contest winner was *OpenTimer* [15], and it is therefore a natural baseline in our evaluation; most other entrants were upgraded Tau14 versions with broadly similar core algorithms, so we do not elaborate on them here. Two papers published the same year, iitRace [20] and MTimer [6], focused on minimizing memory overhead through data-structure reuse. We do not include them in our runtime-focused comparison, because they primarily target memory-footprint reduction and continue to rely on exhaustive DFS/BFS-style searches with quadratic worst-case growth.

After 2015, two notable works extended this line of research. In 2016, HappyTimer [17] introduced a pessimism-removal-during-search block-based algorithm (together with an alternative delay metric), departing from the branch-and-bound tradition and enabling pessimism to be eliminated incrementally during timing updates rather than as a separate preprocessing step; in 2021, ProvablyGoodTimer [11] refined the "pessimism-removal-before-search" paradigm by stripping away one layer of pessimism at a time. Because no public implementations are available, reproducing their results under identical benchmarks, inputs, and scripts would require substantial engineering effort for full reimplementations, we therefore do not include them. Nevertheless, to give a rough sense of scale, ProvablyGoodTimer, which performs better than HappyTimer, reports an average 29× speedup over OpenTimer in its paper [11], which is still below FlashTimer's average 59× over OpenTimer in our experiments.

By contrast, FlashTimer embodies a novel paradigm that is different from both traditional pessimism-removal-before-search and pessimism-removal-during-search paradigms. It performs CPPR only on the $k + \alpha$ paths with the smallest pre-slack values (where k is user-specified) to identify the k most critical paths. Here, α is a small number determined dynamically by our termination check to guarantee correctness, with a theoretical bound provided in Section 5.3. Relative to OpenTimer [14] and to TkTimer [18], this slack-guided search reduces both the number of paths explored and the number of full CPPR operations by orders of magnitude.

Timing Models. Among timing models used to compute pre-slack and pessimism for FlashTimer, the two most common are the early–late model and statistical timing. In our

implementation, we adopt the conventional early–late timing model widely used by STA algorithms, which considers best-case and worst-case delays for each edge, rather than the exact delays shown in the example in Section 3. Accordingly, either the best-case or worst-case delay is used when computing pre-slack and pessimism to ensure soundness. Alternatively, statistical timing models represent edge delays as probability distributions [4]. Because the speedup of FlashTimer stems from exploiting the shallow maximum depth of the clock tree—a property independent of the timing model—we expect FlashTimer to apply to statistical timing as well, although we have not yet implemented that variant.

8 Conclusion

We have presented FlashTimer, which uses cheap pre-slack estimates to prioritize full pessimism removal only on the most promising paths. In our benchmarks, this yields end-to-end speedups of 63× in accuracy mode and 91× in Speed Mode with under 2% accuracy loss. The algorithm runs in $O(n \log n + k \log k)$, where n is the number of circuit nodes and k the number of queried paths, and its soundness guarantees are established in Section 5.1. Extensive experiments on million-gate designs confirm that FlashTimer scales to industrial-sized circuits without compromising timing precision. By effectively removing excess conservatism from timing models while preserving scalability, FlashTimer promises to accelerate design–verify iterations in modern EDA flows. We anticipate its adoption in commercial STA tools—and its extension to statistical timing frameworks—will further streamline timing closure for next-generation chips.

References

[1] Vineeth B and Deepmala Sachan. 2023. How to Maximize Productivity and Lower Cost for Enterprise Prototyping. https://dvcon-proceedings.org/wp-content/uploads/1B2_DVCon_India_2023_Final_Paper_2304.pdf.

[2] Omer Berkman and Uzi Vishkin. 1993. Recursive star-tree parallel data structure. *SIAM J. Comput.* 22, 2 (1993), 221–242.

[3] Jayaram Bhasker and Rakesh Chadha. 2009. *Static Timing Analysis for Nanometer Designs: A Practical Approach.* Springer US, Boston, MA. https://doi.org/10.1007/978-0-387-93820-2

[4] David Blaauw, Kaviraj Chopra, Ashish Srivastava, and Lou Scheffer. 2008. Statistical timing analysis: From basic principles to state of the art. *IEEE transactions on computer-aided design of integrated circuits and systems* 27, 4 (2008), 589–607.

[5] Veena S. Chakravarthi. 2020. Static Timing Analysis (STA). In *A Practical Approach to VLSI System on Chip (SoC) Design.* Springer International Publishing, Cham, 99–116. https://doi.org/10.1007/978-3-030-23049-4_6

[6] Chung-Hao Tsai and Wai-Kei Mak. 2015. A fast parallel approach for common path pessimism removal. In *The 20th Asia and South Pacific Design Automation Conference.* IEEE, Chiba, Japan, 372–377. https://doi.org/10.1109/ASPDAC.2015.7059033

[7] Cadence Corporation. 2026. Timing Closure. https://www.cadence.com/en_US/home/explore/timing-closure.html.

[8] Synopsys Corporation. 2026. What is Static Timing Analysis (STA)? https://www.synopsys.com/glossary/what-is-static-timing-analysis.html.

[9] David Eppstein. 1998. Finding the k Shortest Paths. *SIAM J. Comput.* 28, 2 (Jan. 1998), 652–673. https://doi.org/10.1137/S0097539795290477

[10] Vibhor Garg. 2014. Common Path Pessimism Removal: An Industry Perspective. In *2014 IEEE/ACM International Conference on Computer-Aided Design (ICCAD).* IEEE, San Jose, CA, USA, 592–595. https://doi.org/10.1109/ICCAD.2014.7001412

[11] Zizheng Guo, Mingwei Yang, Tsung-Wei Huang, and Yibo Lin. 2022. A Provably Good and Practically Efficient Algorithm for Common Path Pessimism Removal in Large Designs. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 41, 10 (Oct. 2022), 3466–3478. https://doi.org/10.1109/TCAD.2021.3124758

[12] Jin Hu, Greg Schaeffer, and Vibhor Garg. 2015. TAU 2015 contest on incremental timing analysis. In *2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)* (Austin, TX, USA). IEEE Press, 882–889. https://doi.org/10.1109/ICCAD.2015.7372664

[13] Jin Hu, Debjit Sinha, and Igor Keller. 2014. TAU 2014 contest on removing common path pessimism during timing analysis. In *Proceedings of the 2014 on International Symposium on Physical Design* (Petaluma, California, USA) *(ISPD '14).* Association for Computing Machinery, New York, NY, USA, 153–160. https://doi.org/10.1145/2560519.2565876

[14] Tsung-Wei Huang, Guannan Guo, Chun-Xun Lin, and Martin D. F. Wong. 2021. OpenTimer v2: A New Parallel Incremental Timing Analysis Engine. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 40, 4 (April 2021), 776–789. https://doi.org/10.1109/TCAD.2020.3007319

[15] Tsung-Wei Huang and Martin D. F. Wong. 2015. OpenTimer: A high-performance timing analysis tool. In *2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD).* IEEE, Austin, TX, USA, 895–902. https://doi.org/10.1109/ICCAD.2015.7372666

[16] Tsung-Wei Huang and Martin D. F. Wong. 2016. UI-Timer 1.0: An Ultrafast Path-Based Timing Analysis Algorithm for CPPR. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 35, 11 (Nov. 2016), 1862–1875. https://doi.org/10.1109/TCAD.2016.2524566

[17] Baihong Jin, Guojie Luo, and Wentai Zhang. 2016. A fast and accurate approach for common path pessimism removal in static timing analysis. In *2016 IEEE International Symposium on Circuits and Systems (ISCAS).* IEEE, Montréal, QC, Canada, 2623–2626. https://doi.org/10.1109/ISCAS.2016.7539131

[18] Christos Kalonakis, Charalampos Antoniadis, Panagiotis Giannakou, Dimos Dioudis, Georgios Pinitas, and Georgios Stamoulis. 2014. TK-timer: Fast & accurate clock network pessimism removal. In *2014 IEEE/ACM International Conference on Computer-Aided Design (ICCAD).* IEEE, San Jose, CA, USA, 606–610. https://doi.org/10.1109/ICCAD.2014.7001415

[19] Pei-Yu Lee, Iris Hui-Ru Jiang, Cheng-Ruei Li, Wei-Lun Chiu, and Yu-Ming Yang. 2015. iTimerC 2.0: Fast incremental timing and CPPR analysis. In *2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD).* IEEE, Austin, TX, USA, 890–894. https://doi.org/10.1109/ICCAD.2015.7372665

[20] Chaitanya Peddawad, Aman Goel, Dheeraj B, and Nitin Chandrachoodan. 2015. iitRACE: A memory efficient engine for fast incremental timing analysis and clock pessimism removal. In *2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD).* IEEE, Austin, TX, USA, 903–909. https://doi.org/10.1109/ICCAD.2015.7372667

[21] Reela Samuel. 2023. How to Maximize Productivity and Lower Cost for Enterprise Prototyping. https://community.cadence.com/cadence_blogs_8/b/fv/posts/how-to-maximize-productivity-and-lower-cost-for-enterprise-prototyping.

[22] Thucydides Xanthopoulos. 2009. *Clocking in modern VLSI systems.* Springer Science & Business Media.

[23] Yu-Ming Yang, Yu-Wei Chang, and Iris Hui-Ru Jiang. 2014. iTimerC: Common path pessimism removal using effective reduction methods.

In *2014 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, San Jose, CA, USA, 600–605. https://doi.org/10.1109/ICCAD.2014.7001414